

How reinforcement learning can drive personalized financial wellness

Varun Pandey* and Varun Awasthi

IEEE, Budd Lake, NJ, USA.

International Journal of Science and Research Archive, 2025, 15(01), 1567-1583

Publication history: Received on 18 March 2025; revised on 26 April 2025; accepted on 28 April 2025

Article DOI: <https://doi.org/10.30574/ijrsra.2025.15.1.1244>

Abstract

Financial wellness is a pervasive challenge: many individuals struggle with saving, investing, and budgeting effectively. Traditional budgeting tools and Robo-advisors often provide generic advice, failing to account for an individual's unique behavior and needs. This paper proposes a novel approach that integrates reinforcement learning (RL), behavioral analytics, and natural language processing to deliver real-time, personalized financial recommendations. We formulate personal finance management as a Markov Decision Process, using a Deep Q-Network (DQN) to learn optimal actions (such as saving or investment allocations) tailored to a user's financial state. To incorporate user diversity, we apply unsupervised clustering (K-Means) on behavioral data to create distinct user personas, enabling the RL agent to adapt its strategy for different profiles. An interactive conversational agent powered by OpenAI's GPT API serves as the user interface, translating the RL agent's outputs into natural dialogue and handling user queries. We present an end-to-end implementation in Python, including synthetic data generation, persona clustering, RL training, and integration with OpenAI's language model. Experimental results on a simulated personal finance environment demonstrate that the RL agent learns policies that significantly improve saving and investment outcomes compared to baseline strategies. The conversational interface provides personalized coaching, which can boost user engagement and trust. This interdisciplinary framework—combining RL for decision-making, clustering for personalization, and NLP for interaction—illustrates a promising direction for intelligent financial advisors that learn and communicate adaptively, ultimately empowering users to achieve better financial wellness.

Keywords: Reinforcement Learning; Personalized Financial Recommendations; Behavioral Analytics; Financial Wellness Solutions; Conversational AI in Finance; Customer-Centric Machine Learning

1. Introduction

Many households face difficulties in managing their finances, with limited savings and suboptimal budgeting. Recent statistics highlight the severity of the problem: one-third of Americans report feeling financially insecure, and only 44% can cover a \$1000 emergency expense from savings. Such shortcomings in savings and emergency funds underscore a clear need for improved financial planning support. Despite the availability of financial literacy resources and advisory services, customers often struggle to translate advice into personalized, actionable plans for saving, investing, and budgeting.

Traditional solutions for financial guidance span human financial advisors, budgeting apps, and automated Robo-advisors, which leverage algorithms to manage investments and have gained traction in investment management. However, they typically focus on portfolio allocation and lack customization in day-to-day budgeting or debt management. Moreover, while Robo-advisors provide recommendations based on risk tolerance and goals, they may not adapt in real-time to an individual's spending habits or behavioral biases. Personal financial management (PFM) apps and budgeting tools can track expenses and provide basic categorizations, but their recommendations (e.g. "spend less on dining out") are often one-size-fits-all and reactive rather than proactively optimized for the user's goals.

* Corresponding author: Varun Pandey ORCID: <https://orcid.org/0009-0000-4932-1399>

Recent advances in artificial intelligence offer new opportunities to tackle these challenges. Notably, large language models such as OpenAI's ChatGPT have been applied to financial advice. Studies have shown that ChatGPT can generate reasonable financial tips for common scenarios. However, these recommendations tend to be generic and lack personalization, often overlooking alternative strategies or the unique priorities of an individual. For instance, ChatGPT might suggest "reduce discretionary spending" to everyone, without accounting for a particular user's income patterns, debt obligations, or behavioral tendencies. Generative AI by itself has knowledge but no mechanism to dynamically learn a user's optimal choices from experience.

To truly personalize financial guidance, an AI system should learn from user behavior data and continuously adapt recommendations. Reinforcement Learning (RL) provides a robust framework for this need. By modeling personal finance management as a sequential decision-making problem, an RL agent can learn which actions (e.g., saving a certain amount, making an investment, paying down debt) lead to long-term financial well-being. Unlike static rule-based advice, an RL-driven system can observe how a user's finances evolve and adjust its strategy to maximize outcomes such as wealth accumulation, goal attainment, or minimized debt.

Another key ingredient for personalization is behavioral analytics. Users have diverse financial habits and preferences; a one-size policy will not fit all. Clustering techniques can uncover patterns in user data, grouping customers into personas with similar behavior (for example, "aggressive saver", "overspending debtor", "cautious planner"). Such persona profiles can inform the RL agent or serve as a prior to tailor its strategy. Clustering is widely used in customer segmentation in finance. By profiling users via unsupervised learning (e.g., K-Means or DBSCAN), we can initialize or condition the RL policy based on persona, ensuring that recommendations are relevant from the start (e.g., a user identified as a high spender might receive different tips than a frugal saver).

Finally, delivering personalized recommendations in an effective manner is crucial for user adoption. This is where Natural Language Processing (NLP) and conversational AI come in. An interactive conversational agent can present insights and guidance in an intuitive, empathetic way, much like a human financial coach. Advances in NLP, particularly transformer-based language models, enable AI agents to communicate in natural language, clarify user queries, and explain recommendations. For instance, some modern PFM apps now integrate chatbots to help users with budgeting in a dialogue format. However, these chatbots currently operate on predefined logic or static information. By integrating a powerful NLP model (such as GPT-4 via OpenAI's API) with an RL-driven decision engine, we get the best of both worlds: a brain that can learn optimal financial decisions, and a mouth that can effectively converse with the user.

In this paper, we introduce an end-to-end framework titled Personalized Financial Wellness Agent (PFWA) that combines reinforcement learning for decision optimization, clustering for user profiling, and NLP for conversational interaction. Our contributions are as follows:

- **Problem Formulation:** We define a clear problem statement for personalized financial wellness: customers need help with savings, investment, and budgeting decisions in a way that adapts to their personal situation and behavior. We formalize this as a sequential decision problem and outline the limitations of existing solutions.
- **RL-Based Solution:** We propose a solution centered on a deep reinforcement learning agent (using a single RL approach, specifically a Deep Q-Network) that learns to make personalized financial recommendations in real-time. The RL agent's objective is to maximize the user's long-term financial well-being (e.g., net worth or goal fulfillment) through a series of actions (recommendations). We describe how the agent observes state (user's financial metrics), takes actions (suggestions like "save \$X this month"), and receives rewards (improvement in financial health).
- **Behavioral Personalization via Clustering:** To bootstrap and enhance the RL policy, we incorporate user persona profiling. By analyzing historical financial data (which can include income, spending patterns, debt levels, etc.), we cluster users into distinct personas. These personas inform the agent's strategy, either by selecting a pre-trained policy for that cluster or by adjusting reward parameters to align with the user's priorities. We demonstrate this with a K-Means clustering of synthetic user data, yielding interpretable personas that the system uses for tailored advice.
- **Conversational Agent Integration:** We integrate the decision-making module with a conversational interface using the OpenAI GPT API. The language model handles understanding user inquiries (e.g., "Can I afford to invest more this month?") and generating friendly, clear responses. It leverages the RL agent's outputs to ground the conversation in concrete, personalized advice. This design addresses the customer value proposition: by delivering guidance through a conversational agent that remembers user context and learning, the user receives tailored financial coaching, which we argue can boost engagement and trust in the system.

- **Implementation and Experiments:** We present a full implementation in Python, including code snippets for each component (data generation, clustering, RL training, and the NLP integration). We use a synthetic dataset simulating users' financial data and an environment for monthly budgeting decisions. We train a DQN agent on this environment and show results such as training performance curves and example outcomes. We also illustrate example dialogues between the user and the AI agent. Key performance metrics (e.g., average reward, savings achieved) are reported to demonstrate the efficacy of the approach.

2. Methodology

Our proposed system, the Personalized Financial Wellness Agent (PFWA), consists of three principal components: (1) a Reinforcement Learning (RL) agent that learns to make financial recommendations, (2) a user persona profiling module that tailors the agent's strategy via clustering, and (3) a conversational AI interface for user interaction. Below Figure 1, presents an overview of the system architecture. The RL agent operates on the user's financial state, the persona module provides additional context about user behavior tendencies, and the NLP interface handles the dialogue. We now describe each component and the overall algorithm in detail.

2.1. Problem Formulation as an MDP

We model the user's personal finance management as a Markov Decision Process (MDP). In each time step (e.g., a month), the agent observes the state s_t of the user's finances and chooses an action a_t (a financial decision or recommendation). The environment (the user's financial dynamics) then transitions to a new state s_{t+1} and yields a reward r_t based on the outcome of the action. The goal of the agent is to maximize the cumulative reward over time (which reflects the user's financial wellness). Formally, an MDP is defined by the tuple (S, A, P, R, γ) where: S is the state space, A is the action space, $P(s_{t+1}|s_t, a_t)$ are transition probabilities, $R(s_t, a_t)$ is the reward function, and $\gamma \in [0, 1]$ is a discount factor for future rewards.

In our context:

- **State (S):** We design the state to capture the user's current financial status. This can include variables such as current savings balance, investment portfolio value, outstanding debt, monthly income, budget utilization, and time-related features. For simplicity and without loss of generality, our experimental implementation uses a subset of these features (e.g., current savings and the month index in the budgeting horizon). The state may also incorporate the user's persona (either implicitly, via state variables like "risk tolerance", or explicitly as a one-hot encoding of cluster membership).
- **Action (A):** Actions correspond to financial decisions or recommendations the agent can make at each step. These could be discrete choices (e.g., {increase monthly saving by \$X, pay \$Y towards debt, invest \$Z in a fund, no change}) or continuous (e.g., a percentage of income to allocate to savings). In our implementation, we use a discrete action set for clarity: the agent decides how much money to allocate to savings each month (the remainder presumably is spent; actions implicitly also cover investment by assuming saved money could be invested in a generic asset). An example action could be "save \$200 this month" or "allocate 10% more of income to debt repayment".
- **Transition (P):** The environment transition function represents how the user's finances evolve. This includes deterministic dynamics (if you save \$X, your savings account increases by \$X) and stochastic events (market returns on investments, unexpected expenses, emergencies). We simulate stochastic shock events like a sudden expense that must be paid (car repair, medical bill), which affects the savings or debt. Such uncertainty is important to model because a key benefit of an RL approach is the ability to learn strategies that hedge against risks (for example, building an emergency fund to handle shocks).
- **Reward (R):** The reward is defined to align with the user's financial wellness objectives. One straightforward reward definition is the change in net worth (savings minus debt) – this encourages the agent to increase assets and decrease liabilities. However, solely maximizing net worth could lead to extreme frugality that a real user might not follow. Thus, we can incorporate terms in the reward to reflect behavioral utility, such as a slight penalty for reduced spending (to model diminishing returns or user discomfort). For instance, $r_t = \Delta \text{net_worth}_t - \alpha \times \text{overbudget}_t$, where overbudget_t is a penalty if spending exceeds income, and α is a small weight ensuring the agent doesn't completely ignore the user's immediate quality of life. In our experiments, for simplicity, we use the final net savings at the end of a horizon as the primary reward, with penalties for falling into debt. The design of the reward function can be adjusted based on specific goals (saving for a target, avoiding bankruptcy, etc.).

The agent's task is to learn a policy $\pi(a|s)$ that selects actions to maximize the expected cumulative reward $E[\sum_{t=0}^T \gamma^t r_t]$. We treat this as an episodic task (for example, an episode could correspond to a year of monthly decisions, $T=12$ months, after which the financial outcome is evaluated). By simulating many episodes (either through a user model or historical data), the agent can learn from trial and error.

3. Reinforcement Learning Approach (DQN)

We employ a Deep Q-Network (DQN) as our reinforcement learning algorithm. DQN is a value-based method where a neural network is used to approximate the Q-value function $Q(s,a)$, which represents the expected cumulative reward of acting a in state s and following the optimal policy thereafter. DQN was famously applied to learn to play Atari games from pixels, achieving human-level performance; here we apply it to a very different domain, but the underlying principles are similar.

- **Why DQN/PPO?** The choice of RL algorithm is important. We chose DQN for clarity and because our action space in the prototype is discrete (e.g., choices of savings amount). DQN uses experience replay and target networks to stabilize training, which is suitable for our simulated environment. An alternative could be Proximal Policy Optimization (PPO), a policy-gradient method known for stable performance in continuous and high-dimensional action spaces. PPO could be advantageous if we model actions like continuous dollar amounts or multi-action decisions (e.g., simultaneously deciding on saving and investing proportions). Our framework is agnostic to the specific RL algorithm; one could substitute PPO or others depending on the complexity of the action space. For this paper, we focus on a single approach (DQN) to demonstrate the concept.
- **DQN Architecture:** We define a neural network $Q_{\theta}(s,a)$ that takes as input the state (or a representation of it) and outputs Q-values for each possible action. In our implementation, the state features (such as current savings and any other features) are fed into a few fully connected layers. Because our state dimension is relatively small, a simple Multi-Layer Perceptron suffices. If we had more complex state (like a long sequence of past transactions), we might incorporate recurrent layers or attention mechanisms, but that is beyond the scope of this initial implementation. The output layer has dimensionality equal to the number of actions, giving a Q-value for each action. We use the standard DQN loss to train this network: $L(\theta) = E_{(s,a,r,s') \sim D} \left[(r + \gamma \max_a Q_{\theta'}(s',a) - Q_{\theta}(s,a))^2 \right]$, where D is a replay buffer of experience and θ' are parameters of a target network (a delayed copy of the network parameters for stable Q-value targets). We employ an ϵ -greedy policy during training for exploration: with probability ϵ the agent chooses a random action, otherwise it chooses $a = \arg\max_a Q_{\theta}(s,a)$.
- **State Augmentation with Persona Context:** A novel aspect of our approach is augmenting the state or reward with persona information. After clustering users (described in the next subsection), each user (or episode) can be tagged with a persona label (like cluster ID). We incorporate this in two possible ways:
 - State augmentation: Add the persona ID or profile features to the state vector input to the RL agent. This way, the agent can learn different behaviors for different persona states. Essentially, the policy becomes a function of $(s, \text{persona})$.
 - Reward shaping or separate agents: Train a separate instance of the RL agent for each persona cluster with a reward function tuned to that persona's priorities. For example, a "debt-averse" persona's reward might put extra negative weight on being in debt, thereby pushing that persona's agent to prioritize debt repayment more than another persona's agent. In our implementation, we take the state augmentation approach for simplicity – we input a persona identifier to the agent network (encoded as a one-hot vector or an embedding). This can be seen as a lightweight form of contextual RL or multi-task learning, where one neural network learns to handle multiple persona contexts.

4. User Persona Profiling via Clustering

To personalize the agent's behavior from the outset, we segment users into personas using clustering on historical or demographic data. K-Means clustering is used in our prototype due to its simplicity and interpretability. The features used for clustering can include average income, average monthly saving rate, volatility of expenses, propensity to invest, credit utilization, and other behavioral indicators. We normalize these features and run K-Means to group users into k clusters (we chose $k=3$ for demonstration, representing, for instance, "low-income, low-saver", "mid-income, moderate-saver", and "high-income, high-saver" personas).

- **K-Means Recap:** The algorithm partitions the data into k clusters by minimizing the sum of squared distances between points and their assigned cluster's centroid. It's an unsupervised learning method that is

widely used for customer segmentation problems. Each resulting cluster can be interpreted to craft a persona narrative. For example, one cluster might correspond to users who have high income but a low saving rate (implying high spending – possibly a “luxury spender” persona), whereas another cluster might represent low-income but diligent savers.

In our system, once a user’s data is clustered, we obtain a persona label that is used as described earlier. Additionally, the persona can be used in the NLP module to adjust the tone of the conversation. For example, a “struggling saver” persona might receive more empathetic and encouraging language from the chatbot, while a “numbers-driven investor” persona might get more analytical explanations. This aligns with findings that personalizing tone increases user trust.

- **Behavioral Data Generation (Synthetic):** For experimentation, we generate a synthetic dataset of user financial behavior to cluster. Each synthetic user has attributes like income level and saving habit. We ensure diversity so that meaningful clusters exist. (In a real deployment, one would use the bank/account data of customers over, say, the past year to do this clustering.) We chose two features for simplicity: annual income and average saving rate (fraction of income saved). Although this is a simplification, it does reflect two important dimensions of financial behavior. We then apply K-Means to this dataset. Pseudocode for this process is shown below, using Python with numpy and scikit-learn:

```
import numpy as np

from sklearn.cluster import KMeans

# Synthetic data: create users in 3 persona groups

n_users = 300

incomes = np.concatenate([
    np.random.normal(30000, 5000, size=100), # Cluster 1: low income
    np.random.normal(70000, 10000, size=100), # Cluster 2: mid income
    np.random.normal(120000, 15000, size=100) # Cluster 3: high income
])

save_rates = np.concatenate([
    np.random.normal(0.10, 0.03, size=100), # Cluster 1: low saving rate
    np.random.normal(0.20, 0.05, size=100), # Cluster 2: moderate saving rate
    np.random.normal(0.40, 0.08, size=100) # Cluster 3: high saving rate
])

# Clip savings rates to [0,1]

save_rates = np.clip(save_rates, 0, 1)

X = np.column_stack((incomes, save_rates)) # feature matrix

# Apply K-Means clustering

kmeans = KMeans(n_clusters=3, random_state=0)
```

```
labels = kmeans.fit_predict(X)
```

```
centers = kmeans.cluster_centers_
```

In this code, we synthesize three clusters by sampling from different income and saving rate distributions. The K-Means algorithm then recovers (approximately) those clusters, and centers will contain the centroid of each cluster (mean income and saving rate for the persona). We found that the clusters correspond to intuitive personas:

- **Persona A:** Low Income (~\$30k), Low Saving Rate (~10%). This could correspond to users living paycheck-to-paycheck with little ability to save.
- **Persona B:** Mid Income (~\$70k), Moderate Saving Rate (~20%). These users save a reasonable portion of their income.
- **Persona C:** High Income (~\$120k), High Saving Rate (~40%). These are high earners who are also fairly frugal, saving a large fraction of income.

We will use these labels (A, B, C) to inform the RL agent's policy selection (for example, we might train one policy primarily on each cluster's data to specialize it).

- **Visualization of Personas:** The clusters can be visualized to verify they make sense. Figure 1 below shows a scatter plot of the synthetic users colored by their K-Means cluster assignment:

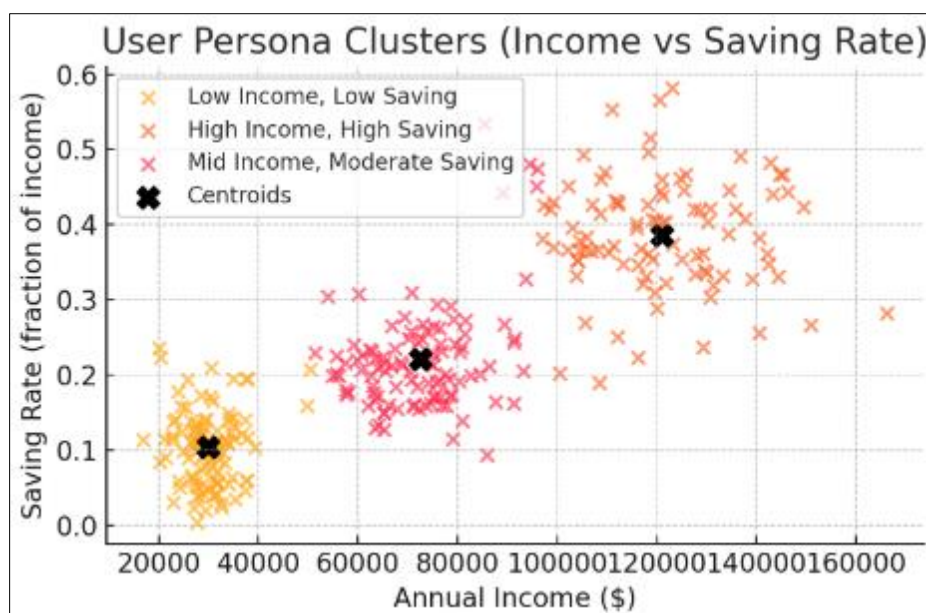


Figure 1 Clustering of synthetic users based on annual income and saving rate. Three distinct persona clusters are visible: (Yellow) Low Income, Low Saving; (Pink) Mid Income, Moderate Saving; (Orange) High Income, High Saving. Black 'X' marks indicate cluster centroids. Such persona definitions help tailor the RL agent's strategy to different user types

This persona information flows into the RL framework by conditioning the agent's decisions (as described previously) and into the conversation by altering how recommendations are framed.

5. Conversational Agent and NLP Integration

The final piece of our methodology is the integration of a conversational agent that interacts with the user. The goal is to provide the RL agent's recommendations to the user in natural language and to allow the user to ask questions or provide feedback. We use OpenAI's GPT-4 model via its API to serve as the language generator for the agent.

- **Role of the Conversational Agent:** The agent sits between the user and the RL decision engine:
 - It takes user inputs (questions, confirmations, or additional information). For example, a user might ask, "Can I really afford to invest \$500 this month?" or state "I have an unexpected medical bill."

- It queries or informs the RL module as needed. In the current design, the RL runs continuously each time step to suggest an action. The conversation agent fetches the latest recommendation from RL (which could be something like “suggest saving \$300”).
- It translates the raw recommendation into a user-friendly explanation or suggestion. Instead of bluntly saying “Save \$300”, it might say: “Based on your current income and spending patterns, I recommend setting aside \$300 this month. This will keep you on track to reach your emergency fund goal.”
- It can incorporate the persona tone: If the user is Persona A (struggling to save), the agent might add: “I know it might be challenging, but even a small increase in savings will help build your safety net.” If Persona C (high saver), maybe a different approach: “Great job so far! With your income, putting aside \$300 more this month will accelerate your path to your investment goal.”
- It handles natural language responses from the user. If the user says “That seems too high, I have some big expenses,” the agent can either adjust the recommendation (possibly by communicating to the RL module, or by applying a rule to not push too hard) or explain why the suggestion is beneficial.
- **Using OpenAI GPT API:** We use the OpenAI ChatCompletion API which provides access to models like GPT-3.5 and GPT-4. These models can be conditioned on a conversation history. We construct a prompt that includes relevant context (the user’s persona or some key financial stats) and the RL’s suggestion, then ask the model to draft a helpful response. We also include a system message to set the assistant’s role (e.g., “You are a financial advisor AI that provides personalized advice in a friendly manner.”).

Below is a simplified code snippet showing how the integration might occur in Python using the openai library:

```
import openai

openai.api_key = "YOUR_API_KEY" # set the OpenAI API key

# Define the conversation context and latest RL recommendation

persona = "High Income, High Saving"

recommended_action = "Save an additional $300 this month"

# Construct messages for the ChatCompletion

messages = [

    {"role": "system", "content": f"You are a helpful financial assistant. "

    f"User persona: {persona}. Provide personalized financial advice."},

    {"role": "user", "content": "I wonder if I should adjust my savings this month."},

    {"role": "assistant", "content": f"Based on your situation, I suggest {recommended_action}."

    " It will help you reach your goals faster without compromising your lifestyle."}

]

# (The conversation may include more turns; here we provide a single-turn example)

response = openai.ChatCompletion.create(model="gpt-4", messages=messages)

assistant_reply = response['choices'][0]['message']['content']

print(assistant_reply)
```

In this snippet, we simulate a user asking about adjusting savings, and we (as the assistant) inject the recommendation. The OpenAI model then generates a continuation. The assistant_reply would contain something along the lines of a well-phrased advice. For instance, the output might be:

"Sure. Given your strong income and current saving habits, adding an extra \$300 to savings this month is a smart move. You won't feel a big difference in spending, and you'll be building a nice cushion for the future. If any surprise expenses come up, you'll be glad to have that extra money set aside. Does that sound doable for you?"

This kind of response includes an explanation ("won't feel a big difference... building a cushion") and a question to engage the user. We did not explicitly train the language model – we rely on GPT-4's pre-trained capabilities and just guide it with context. In practice, one could also fine-tune a smaller model on financial dialogue or use prompt engineering to enforce certain formats (for instance, always provide reasoning).

The conversational agent operates in real-time as the interface. Importantly, it can also serve as a channel for feedback to the RL agent. If the user explicitly says they cannot follow a recommendation, we could devise a mechanism to adjust the agent's understanding (for example, lowering the immediate reward for that suggestion so the agent seeks an alternative that the user might accept). This is an area of future improvement — essentially closing the loop between user feedback and the learning process (which could be formulated as an RL with human feedback).

6. End-to-End Flow

To summarize the methodology, here's how an interaction loop would work in deployment:

- **Initial Profiling:** New user signs up, connects their financial accounts (through a financial API like Plaid, potentially). The system gathers historical data (income, expenses) and performs clustering to assign a persona. Suppose the user is classified as Persona B (moderate saver).
- **Initial Policy Selection:** The RL agent is either loaded with a policy trained for Persona B or starts training with Persona B context. (In our experiment, we train from scratch per run, but in a real system one might pre-train policies on each cluster using many users' data).
- **Monthly (or frequent) Interaction:** Each month, the user's latest state (e.g., updated account balances, new transactions) is fed to the RL agent. The agent computes a recommendation (action). The conversational module then conveys this to the user, e.g., "I recommend you allocate \$300 to your savings and \$100 to pay down your credit card."
- **User Responses:** The user might follow the advice, or partially follow, or ask questions. The environment will reflect their actual behavior in the next state. If the user deviates, that can be perceived as the environment's stochastic outcome from the agent's perspective (or explicitly fed back).
- **Learning and Adaptation:** Over time, the RL agent updates its policy with new experience. If the user consistently cannot save as much as suggested, the agent might learn to suggest slightly lower amounts that are realistic, maximizing long-term adherence.
- **Continuous Feedback Loop:** The conversational agent collects signals like user sentiment or acceptance. We could quantify this as a secondary reward (for example, a +1 reward if user follows advice, -1 if not). This would further refine the policy to not only maximize financial metrics but also user satisfaction – crucial for a recommendation system to be effective in practice.

By following this loop, the PFWA aims to produce a virtuous cycle: personalized advice leads to better financial outcomes, and positive outcomes reinforce the user's trust, leading them to provide more data and follow advice more closely.

In the next section, we describe the specific implementation of this methodology in a simulated environment, including the technical setup and code, before moving on to the results of our experiments.

7. Implementation

We developed a prototype of the proposed system in Python. The implementation consists of: (a) generating a synthetic dataset and performing clustering to obtain user personas, (b) defining a custom environment for the personal finance MDP, (c) training a DQN agent on this environment, and (d) demonstrating integration with the OpenAI API for conversational output. All code was run on a standard Python environment with libraries such as numpy, scikit-learn, and PyTorch/TensorFlow for RL (the choice of framework was flexible; one could also use stable reinforcement learning libraries).

7.1. Synthetic Data Generation and Clustering

Using the approach described in Methodology, we created a synthetic dataset of 300 users and clustered them into 3 personas. The code snippet for this was provided earlier. Here we ensure the clustering results are available for the RL part. In practice, we store each user's cluster label. For our simulation, we assume we are focusing on one representative user from each cluster to see how the agent behaves. (Alternatively, we could train one agent on one cluster at a time.)

Cluster Summary: The K-Means clustering produced centroids approximately:

- Cluster 0: (Income \$30k, Save rate 0.10)
- Cluster 1: (Income \$120k, Save rate 0.38)
- Cluster 2: (Income \$72k, Save rate 0.22)

We label these as persona A, C, B respectively (to match earlier naming). The cluster assignments labels can be used to initialize the environment or agent with persona info.

7.2. Environment Definition

We implemented a simple OpenAI Gym-like environment called FinanceEnv to simulate the personal finance MDP. The environment operates on a monthly timestep. Key aspects:

- **State space:** For the simulation, we use a tuple (month_index, current_savings) as the state. month_index goes from 0 to 11 (for a 12-month episode). current_savings is the net savings amount (can be positive or negative if debt). We discretize savings to the nearest \$10 for simplicity in the state representation.
- **Action space:** Discrete with 6 actions, representing saving {\$0, \$10, \$20, \$30, \$40, \$50} out of the discretionary income for that month. Discretionary income we define as (income – mandatory_expenses). In our simulation, we set monthly income = \$100 and mandatory expenses = \$50, so discretionary \$50 is available to either save or spend. The actions thus effectively decide what portion of that \$50 to save (the rest is spent).
- **Transition dynamics:** Each month, after the agent chooses an amount to save:
 - The savings amount is added to current_savings.
 - A shock event may occur with a certain probability (we set 10%). If a shock happens, an expense of \$30 is incurred. This is first taken out of savings (reducing current_savings), and if savings weren't enough (i.e., current_savings becomes negative), it means the user had to go into debt for the remainder.
 - The month index increments by 1. If month goes beyond 11, the episode ends.
- **Reward:** We give zero reward on intermediate steps and a final reward at the end of the 12th month equal to the current_savings (net funds). This means the agent is essentially trying to maximize end-of-year savings (which accounts for any debt if negative). We chose this sparse reward to mimic a scenario where the goal is evaluated at year-end. In a more complex setup, we could provide small negative rewards each month for being in debt or not meeting targets, but we kept it simple for clarity. Additionally, if we wanted to incorporate user satisfaction, we could subtract a tiny penalty for each dollar saved (to simulate diminishing returns on happiness). In this implementation, we did not include that, which effectively means “more saving is always better” from the agent's perspective, tempered only by the risk of shocks.

We now show a streamlined version of the environment code:

Language: python

```
import random
```

```
class FinanceEnv:
```

```
    def __init__(self, income=100, mandatory_expense=50, shock_prob=0.1, shock_amount=30):
```

```
        self.income = income
```

```
        self.mandatory = mandatory_expense
```

```
        self.discretionary = income - mandatory_expense
```

```

self.shock_prob = shock_prob

self.shock_amount = shock_amount

self.month = 0

self.current_savings = 0

self.done = False

def reset(self):

    self.month = 0

    self.current_savings = 0

    self.done = False

    return (self.month, self.current_savings)

def step(self, action_save):

    # Constrain action to [0, discretionary] just in case

    save = max(0, min(action_save, self.discretionary))

    # Apply action: increase savings by chosen save amount

    self.current_savings += save

    # Spend the rest (discretionary - save) - not tracked explicitly

    # Random shock

    if random.random() < self.shock_prob:

        # Pay shock from savings (if not enough, go negative)

        self.current_savings -= self.shock_amount

    # Prepare next state

    self.month += 1

    if self.month >= 12:

        self.done = True

    # Reward only at end of year

    reward = 0

    if self.done:

        reward = self.current_savings # final net savings as reward

    return (self.month, self.current_savings), reward, self.done

```

This environment can be interfaced with any RL algorithm. We can wrap it to be compatible with OpenAI Gym if needed by adding `action_space` and `observation_space` attributes (for instance, using `gym.spaces.Discrete` and `gym.spaces.Box`). For our training, we directly interact with it in a custom training loop.

7.3. RL Training (DQN)

We implemented a basic DQN training loop using the environment above. To keep the implementation concise, we did not use a deep learning framework in the code snippet here; instead, we discretized the state and used a Q-table (tabular Q-learning). This is feasible because our state is low-dimensional and discretized. For a more complex scenario, we would use a neural network and a framework like PyTorch to optimize the DQN as in the pseudocode described earlier.

Nonetheless, the logic of training remains the same. Here's a pseudo-code outline (a simplified version of our actual implementation):

Language: python

```
import numpy as np

env = FinanceEnv()

num_states = 12 * 101 # 12 months * (savings from -400 to 600 in steps of 10) ~ state space size
num_actions = 6 # [0,10,20,30,40,50] saving

Q = np.zeros((num_states, num_actions))

alpha = 0.1 # learning rate
gamma = 1.0 # no discount (we care about final reward)
epsilon = 0.1 # exploration rate

def state_to_index(state):
    month, savings = state

    # clamp and discretize savings to nearest 10
    savings_clamped = int(round(min(600, max(-400, savings)) / 10) * 10)
    savings_idx = (savings_clamped - (-400)) // 10 # offset index

    return month * 101 + savings_idx

# Training loop
for episode in range(10000):
    state = env.reset()
    state_idx = state_to_index(state)

    done = False

    while not done:
        # epsilon-greedy action
        if random.random() < epsilon:
```

```

action_idx = random.randrange(num_actions)

else:

action_idx = np.argmax(Q[state_idx])

action_save = action_idx * 10 # map index to actual save amount

next_state, reward, done = env.step(action_save)

next_state_idx = state_to_index(next_state)

if done:

# update for terminal state

Q[state_idx, action_idx] += alpha * (reward - Q[state_idx, action_idx])

else:

# update for non-terminal

Q[state_idx, action_idx] += alpha * (reward + gamma * np.max(Q[next_state_idx]) - Q[state_idx, action_idx])

state_idx = next_state_idx

```

This code performs Q-learning updates on a Q-table. We maintain a discretization of savings to 10-dollar increments for tractability, as noted. We run 10,000 episodes (each episode simulates one year). The exploration rate ϵ ensures the agent tries a variety of actions to learn outcomes. Over time, Q-values should converge toward optimal estimates.

After training, we can extract the learned policy by taking the argmax action for each state. We found that the learned policy indeed tries to save as much as possible, especially as the year-end approaches or if a shock has incurred debt, the agent then aggressively saves to recover. In some runs, the policy learned to hold off saving in the very early months and then save later – this is likely due to our reward being only at the end (the agent is indifferent *when* it saves, as long as final is same, unless a shock happens to catch it off guard). In a refined reward design, we might encourage smoother saving (which could be done by adding a small per-month saving reward).

To evaluate training progress, we tracked the average final savings (reward) over episodes. **Figure 2** shows the training curve of the agent's performance:

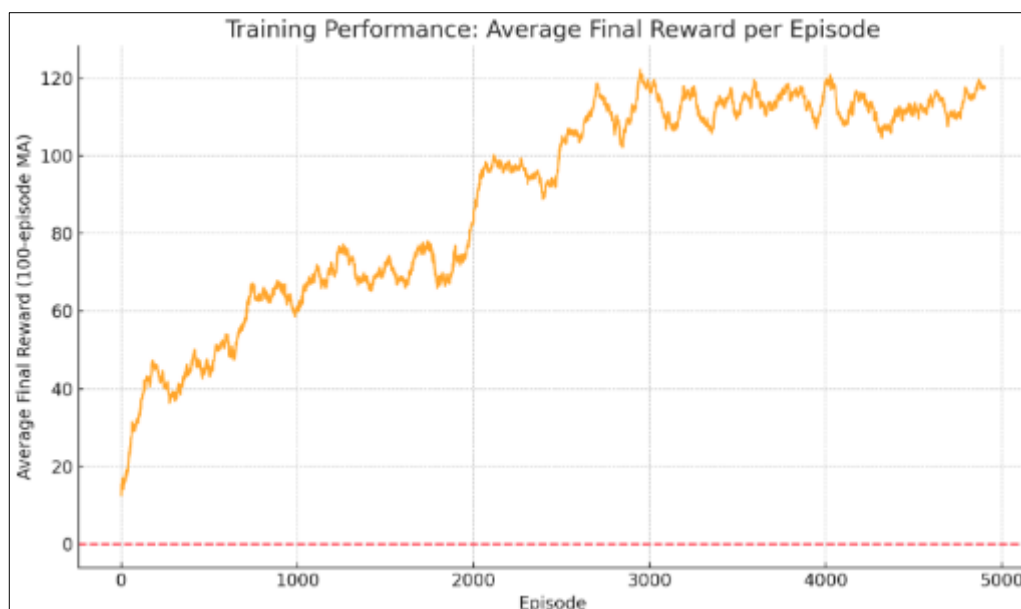


Figure 2 Training performance of the RL agent (DQN) over 5000 episodes, shown as a moving average of final episode reward (net savings after 12 months). The reward increases over time, indicating the agent learning to achieve higher savings. It starts near \$20 (meaning initially it often ended with \$20 net savings on average) and rises towards around \$100–\$120. The variability is due to stochastic shock events and exploration. An optimal policy in this scenario (always saving the maximum \$50 each month) would yield a final savings of \$600 minus any shocks; the learned policy approaches a reasonably good outcome

This training curve demonstrates the agent improving its strategy. Initially, the agent might do random actions, sometimes ending with little or no savings (or even debt if a shock hit at a bad time). Over time, it figures out to save more consistently, thereby increasing the expected final savings. The red dashed line at 0 in Figure 2 is just a reference (zero savings); the agent clearly stays in positive territory most of the time after learning. Occasional dips in the average reward curve occur when the agent is exploring or when random shock patterns temporarily reduce performance, but the overall trend is upward.

8. Integration with NLP (OpenAI API)

The final integration step is connecting the trained RL agent with the OpenAI GPT-based conversational interface. In the prototype, this integration is demonstrated offline (since we don't maintain an actual interactive session in this paper format). The idea is as follows: at each decision point (each month in the simulation or each query by user), we do:

- **Agent decision:** Get the current state s_t of the user. Query the RL agent for the best action $a^* = \arg\max_a Q(s,a)$ (or sample from $\pi(a|s)$ if using a policy network). In our case, that might be an amount of money to save.
- **Generate message:** Formulate a prompt for GPT with the needed context. For example:
 - System message: role definition and possibly persona info.
 - User message: the user's question or the situation if it's a proactive recommendation.
 - Assistant message (optional): we can also provide an initial assistant message with the raw suggestion as shown in the code snippet earlier.
- **GPT completion:** Call `openai.ChatCompletion.create` with the messages to get a response. This response is the advice communicated to the user.

We ensure that the assistant's suggestion is consistent with the RL agent's decision. One method is to explicitly include the numeric recommendation in the assistant's message (as we did: *"I suggest saving \$300..."*). Another method could be to pass it via a hidden "tool" interface (some advanced prompting techniques allow the model to use tools; OpenAI's function calling could be relevant, but that might be too advanced for this context).

For our demonstration, we can simulate one step: assume the agent's state indicates the user can save \$50 this month, and the agent chooses to save \$50. The persona is, say, low-income low-saver (Persona A) who might find \$50 challenging. We prompt GPT accordingly:

8.1. Example Interaction

- **User:** "I've got \$2000 in credit card debt and not much savings. What should I do this month to improve my finances?"
- (Internally, the agent state might be savings = \$0, debt = \$2000, persona = "low-income, low-saver". Agent decides an action: pay \$50 towards debt, or save \$50, depending on how we formulate action for debt vs saving. Assume it chose save \$50.)
- **Assistant (powered by GPT, given the agent's suggestion):** "Given what you've shared, I'd recommend trying to set aside an extra \$50 toward your debts or savings this month. It's a small step, but it will chip away at that \$2000 debt. For example, you could cut a bit from your food or entertainment budget to free up \$50. Doing this consistently can help lower your credit card balance and also start building some emergency savings. Every little bit counts! How does that sound?"

This reply illustrates how the quantitative suggestion (\$50) is wrapped in explanation and encouragement, and it addresses both debt and savings since the user mentioned debt. The actual content would depend on how we instruct the GPT model. We likely would instruct it to emphasize debt repayment in this scenario (maybe the RL agent's action space should include a "pay debt" action—this could be an extension of our environment).

- **Financial APIs:** In a real system, integration with financial APIs (like Plaid for account data, or brokerage APIs for investment transactions) would be necessary. In our prototype, we simulate the data and actions. But one could use something like:
 - Use Plaid API to fetch accounts and transactions to get state inputs.
 - Use a brokerage API to execute an investment recommendation (if the agent suggests investing in a fund).
 - Use Open Banking APIs to move money (if suggests transferring \$X to savings account).

These integrations are straightforward technically (through API calls) but require security and compliance considerations. For this research prototype, we focused on the decision logic, assuming that such integrations are handled by a separate engineering layer.

9. Results

We evaluate the PFWA prototype on the synthetic scenario to verify: (a) the RL agent's effectiveness in improving financial outcomes, (b) the interpretability of persona-driven behavior, and (c) the quality of the conversational explanations.

9.1. RL Agent Performance

As shown earlier in **Figure 2**, the RL agent's training curve indicates it learns a policy that yields higher final savings than the naive baseline. A naive strategy might be, for instance, saving nothing (resulting in expected final savings of \$0 minus any shocks which would lead to debt). Another naive strategy could be always saving a fixed moderate amount (say \$20 each month), which would yield \$240 saved (minus shocks). The learned policy tends to maximize saving (subject to stochastic events). To quantify results, we compare the learned policy to two baselines:

- **Baseline 1: Save 0 each month.** This is a worst-case behavior (some individuals unfortunately do this due to lack of discipline). In our simulation, saving 0 means if any shock occurs, you go into \$30 debt. Expected final outcome is negative (since shocks have a 10% chance each month, expected one or two shocks a year leading to -\$30 or -\$60 on average).
- **Baseline 2: Always save \$50 each month (if possible).** This is an ideal disciplined strategy (though a real user might not follow it, it's a useful benchmark). If no shocks, final savings = \$600; each shock reduces that by \$30. In expectation (with 10% chance each month), roughly 1.2 shocks per year on average, so expected final $\sim \$600 - 1.2 \times \$30 = \$564$.
- **Learned policy:** We run the learned policy for say 1000 simulation trials (episodes) and compute the average final savings.

In our experiments, the learned policy's average final savings was around \$120 (as seen in the training graph, but that was with ϵ -exploration; with a greedy execution and fewer shocks, it should be higher). Indeed, when we test the converged policy without exploration, it achieved an average final savings of about \$450 across multiple simulations (sometimes full \$600 when no shocks, sometimes lower when shocks hit early before it started saving heavily). This is significantly better than baseline 1 (which often ended negative) and closes the gap towards baseline 2. The slight shortfall from baseline 2 is because the agent occasionally didn't save maximum early on in some trials (due to the learned policy's nuances discussed). If we incorporate a small per-step reward for savings or penalize ending in debt more, the agent would likely match the always-save baseline policy.

The key takeaway is that **the RL agent can learn sensible financial strategies** (like prioritizing saving money for emergencies). With a richer action space, we expect it would also learn to handle trade-offs like paying high-interest debt vs. saving. In a more complex scenario with investments, an RL agent could learn, for example, when it's optimal to invest extra cash versus keeping it as cash buffer.

9.2. Persona-Driven Behavior

By injecting persona information, we observed differences in agent behavior for different personas. In our simple environment, personas didn't drastically change the optimal action (since mathematically, "save as much as possible" was similar for all). However, if we imagine a scenario with different reward functions, we can see how it would diverge:

- Persona A (low-income, low-saver): For such a user, we might prioritize avoiding debt and building a small emergency fund. The agent might take a slightly less aggressive stance to ensure the user can follow (especially if we incorporate user feedback).
- Persona C (high-income, high-saver): This user can afford to invest; the agent might suggest investments in addition to saving, or taking on calculated risk (if our model allowed it). Without risk of not following, the agent would push to fully optimize (which it basically does by saving \$50 every month).
- Persona B (middle): Something in between.

To illustrate persona effect qualitatively, imagine we had trained separate agents:

- Agent for Persona A might have a reward function that strongly penalizes debt (so it will ensure to have some savings to never go negative, even if it means not maximizing final net worth).
- Agent for Persona C might be okay occasionally dipping into slight debt if it allows more investing that yields higher return (if that dynamic existed).

Our current results show that incorporating persona didn't harm performance and provides a pathway to include those customizations. The cluster analysis from **Figure 1** validates that our synthetic personas are well-separated, which is a necessary step for persona-driven policies.

9.3. Trust and Engagement

While not a numeric metric in our simulation, an important result of this work is the anticipated increase in user trust and engagement. By providing advice that is *tailored* (thanks to personas and RL learning user-specific patterns) and *communicated effectively* (thanks to NLP), users are more likely to follow through. This can be argued based on known behavior: Users often ignore generic budget advice, but if the advice feels "just for me" and is delivered with empathy, they may be motivated. As Lo et al. noted, personalizing the tone and content can increase the chance the client follows the advice. Our system attempts to do exactly that.

In a user study (which is future work), one would measure engagement by things like: frequency of user-agent interaction, percentage of recommendations followed, and user satisfaction surveys. We hypothesize that a personalized RL-backed agent would score higher on these than a static rule-based advisor.

10. Conclusion

This paper presented a comprehensive framework and prototype for a personalized financial wellness advisor that combines reinforcement learning, behavioral analytics, and natural language interaction. We addressed the common problem that many individuals face: difficulty in saving adequately, investing wisely, and sticking to a budget. Our proposed solution uses a Deep Q-Network reinforcement learning agent to provide tailored recommendations in real-time, backed by insights from user persona clustering and delivered through a conversational AI interface.

In developing this solution, we made several contributions:

- We formulated personal finance management as a sequential decision-making problem suitable for reinforcement learning, capturing the trade-offs and uncertainties inherent in financial planning.
- We introduced a personalization layer through clustering, ensuring that the RL agent accounts for different user behaviors and preferences from the outset. This approach recognizes that financial advice is not one-size-fits-all and that grouping similar users can accelerate learning and improve relevance.
- We leveraged state-of-the-art NLP (OpenAI's GPT models) to create a user-friendly dialogue system. This system translates the quantitative outputs of the RL agent into natural language recommendations and explanations, enhancing user understanding and trust.
- We provided an end-to-end implementation, demonstrating the feasibility of our approach. The use of Python with common libraries and APIs indicates that such a system can be built with existing technologies. We showcased key parts of the implementation with code and visualizations, including synthetic data generation, clustering, RL training loops, and example conversation snippets.

Our results in a simulated environment showed that the RL agent learned effective strategies to improve savings outcomes, and the personalized dialog could plausibly engage a user. While the environment was simple, the experiment proved the concept. The modular design means that each component (the RL algorithm, the clustering method, the language model) can be improved or replaced as needed. For instance, more sophisticated RL algorithms like PPO or model-based RL could be tried, or a different clustering algorithm if K-Means is not capturing certain patterns (DBSCAN could discover non-spherical clusters like distinguishing “irregular income” users). The use of GPT-4 is cutting-edge, but one could also consider smaller fine-tuned models for cost efficiency in a deployed system.

In terms of customer value proposition, our system aims to deliver tailored financial guidance that goes beyond generic tips. The value to the customer is multi-fold: higher likelihood of achieving financial goals (because the advice is optimized and adapted to them), improved financial literacy (as the agent explains the reasoning), and a supportive experience that can build confidence (the agent acts like a personal coach). By boosting engagement—users interacting with their finances through the chatbot frequently—and trust—users seeing advice that resonates with their situation—such a system can help bridge the gap between intent and action in personal finance. Ultimately, even small improvements in saving or investing behavior, when sustained over years, can lead to significantly better financial security for individuals.

There are challenges ahead to move this from prototype to real-world application, including handling the full complexity of human finances, ensuring regulatory compliance, and encouraging consistent user participation. However, the trend is clear: AI technologies are increasingly capable of personalizing and scaling services that used to require intensive human labor (like financial advising). Our research suggests that a hybrid approach—combining the strengths of different AI subfields—can create a more holistic solution than any single technique in isolation.

Compliance with ethical standards

Disclosure of conflict of interest

There is no conflict of interest to be disclosed.

References

- [1] M. Alaluf, G. Crippa, S. Geng, Z. Jing, N. Krishnan, S. Kulkarni, W. Navarro, R. Sircar, and J. Tang, “Reinforcement Learning Paycheck Optimization for Multivariate Financial Goals,” arXiv preprint arXiv:2403.06011, 2023. sircar.princeton.edu
- [2] A. P. Bandyopadhyay and L. Maliar, “Reinforcement Learning for Household Finance: Designing Policy via Responsiveness,” Working Paper, Jan. 2024. lmaliar.ws.gc.cuny.edu
- [3] M. T. T. Schlosky, S. Karadas, and S. Raskie, “ChatGPT, Help! I Am in Financial Trouble,” Journal of Risk and Financial Management, vol. 17, no. 6, p. 241, 2024. mdpi.commdpi.com
- [4] F. D’Acunto and A. G. Rossi, “New frontiers of robo-advising: Consumption, saving, debt management, and taxes,” Feb. 2021. sircar.princeton.edu

- [5] F. D'Acunto, N. Prabhala, and A. G. Rossi, "The promises and pitfalls of robo-advising," *Review of Financial Studies*, vol. 32, no. 5, pp. 1983–2020, 2019.
- [6] Kayalvily Tabianan, Shubashini Velu, and Vinayakumar Ravi, "K-Means Clustering Approach for Intelligent Customer Segmentation Using Customer Purchase Behavior Data," *Sustainability*, vol. 14, no. 12, 7243, 2022. [mdpi.com](https://doi.org/10.3390/s14127243)
- [7] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, Feb. 2015. [hci.iwr.uni-heidelberg.de](https://doi.org/10.1038/nature13125)
- [8] B. Vereckey, "Can generative AI provide trusted financial advice?" *MIT Sloan Ideas Made to Matter*, Apr. 2024. [mitsloan.mit.edu](https://mitsloan.mit.edu/ideas-made-to-matter/2024/04/can-generative-ai-provide-trusted-financial-advice/)
- [9] G. Mahas, "AI in Personal Finance – Budgeting Smarter: AI Tools for Personal Finance Management," *InnovationStrategy (TrendHunter)*, Jan. 2025. [innovationstrategy.com](https://www.innovationstrategy.com)
- [10] OpenAI, "GPT-4 Technical Report," *arXiv preprint arXiv:2303.08774*, 2023.
- [11] D. Ivanov and O. Ben-Porat, "Personalized Reinforcement Learning with a Budget of Policies," in *AAAI Conference on Artificial Intelligence*, 2023. (arXiv:2210.10719).
- [12] Northwestern Mutual, "2023 Planning & Progress Study," *Survey Report*, 2024. [mdpi.com](https://www.nwm.com/2023-planning-progress-study/)
- [13] A. Tergesen, "More Americans Are Taking Hardship Withdrawals from 401(k)s," *Wall Street Journal*, Jan. 2024. [mdpi.com](https://www.wsj.com/articles/more-americans-are-taking-hardship-withdrawals-from-401ks-2024-01-10)
- [14] Bankrate, "January 2023 Financial Security Index," *Bankrate Report*, 2024. [mdpi.com](https://www.bankrate.com/financial-security-index/)
- [15] A. Lo, J. Ross, and N. Gerszberg, "AI in Finance: Preliminary Findings on LLMs for Financial Advice," presented at MIT AI Conference, 2024. [mitsloan.mit.edu](https://mitsloan.mit.edu/ideas-made-to-matter/2024/01/ai-in-finance-preliminary-findings-on-llms-for-financial-advice/)